TRADESON PLATFORM

Tech Stack Decision Guide & Pre-TDD Discovery

Phase 1: Web App (MVP) → Phase 2: Native App (Beta / iOS)

Version 1.0 | Pre-Technical Design Document Input

1. Purpose of This Document

This document serves as the bridge between the TradesOn PRD and the Technical Design Document (TDD). Before the TDD can be written, the team needs to make binding decisions on the technology stack for each delivery phase. This guide presents the options you are considering, compares them across cost, speed, complexity, and team familiarity, and surfaces the specific discovery questions and dependency decisions that must be resolved before engineering begins.

The document is organized around three outputs:

- A decision comparison matrix across your candidate stacks (Section 2)
- A full tech stack options analysis by layer — cost, tradeoffs, and recommendation (Section 3)
- Pre-TDD discovery questions, design decisions, and requirement dependencies by phase (Section 4)

2. Stack Scenario Comparison Matrix

Four candidate stack scenarios are mapped below based on your stated preferences (Lovable, Cursor, MindStudio, Flutterflow, GCP/Vertex AI, Supabase, n8n). Each scenario is scored across the dimensions most critical to your context: development speed, cost at MVP scale, team familiarity, scalability ceiling, and vendor risk.

●●●	Fast / Low Cost / High Fit	●●○	Moderate	●○○	Slow / High Cost / Low Fit	⚠	Risk / Not Recommended	—	Not applicable	Legend	Score Guide

Dimension	Scenario AFull Low-Code(Lovable + MindStudio+ Supabase)	Scenario BGCP-Native(Vertex AI + ADK+ Cloud Run + Firebase)	Scenario CHybrid Preferred(Lovable/Cursor + n8n+ Supabase + Vertex AI)	Scenario DFull GCP(Vertex AI + ADK+ Cloud SQL + Firebase+ GCP hosting)	Scenario EFlutterFlow MVP(Skip web, go directto iOS)	Scenario FCursor + Supabase(Code-first web,no low-code UI)	Notes
Dev Speed (Web MVP)	●●●	●○○	●●●	●○○	●●○	●●○	Low-code tools (Lovable, FlutterFlow) dramatically cut frontend time. GCP-native requires infra setup.
Dev Speed (Native iOS)	●○○	●●○	●●○	●●○	●●●	●○○	FlutterFlow is fastest path to iOS. Lovable is web-only. Cursor needs React Native setup.
Monthly Cost at MVP Scale (<500 users)	●●●	●●○	●●●	●○○	●●○	●●●	Supabase free tier + Lovable hobby plan ≈ $50-100/mo. Full GCP can exceed $300-500/mo at small scale.
Monthly Cost at Growth Scale (5K-50K users)	●●○	●●●	●●●	●●●	●●●	●●●	GCP scales better at volume. Supabase Pro handles mid-scale well.

							Low-code per-seat costs rise.
Team Familiarity (Kevin)	●●●	●●●	●●●	●●●	●●○	●●○	GCP is your home turf. Supabase + n8n already in use for HomeDeetz. Cursor is new but accessible.
AI Agent Capability(Job Intake, Compliance)	●○○	●●●	●●●	●●●	●○○	●●○	MindStudio has limits for complex multi-step agents. Vertex AI ADK is purpose-built for agent pipelines.
Vendor Lock-in Risk	⚠	●●○	●●○	●○○	⚠	●●●	Lovable and FlutterFlow are proprietary. Code export from FlutterFlow is possible but imperfect. GCP is manageable with good architecture.
Compliance Workflow Support(Doc verify, AI review)	●○○	●●●	●●●	●●●	●○○	●●○	Needs real orchestration (n8n or ADK). MindStudio too lightweight for multi-step compliance chains.
VPC/Instance Stability	●●●	●○○	●●○	●○○	●●●	●●●	You flagged GCP VPC instability. Serverless (Cloud Run, Supabase) reduces this. Raw VM instances are the problem, not GCP itself.
Scalability Ceiling	●○○	●●●	●●●	●●●	●●○	●●●	Low-code UIs (Lovable, FlutterFlow) hit walls at complex custom logic. Code-first or

							hybrid is safer long-term.
Time to First Demo	●●●	●○○	●●●	●○○	●●○	●●○	Lovable + MindStudio or Lovable + n8n can produce a clickable demo in days.
Path to iOS App	⚠	●●○	●●○	●●○	●●●	●●○	Lovable has no native app output. FlutterFlow is the clearest path to iOS without a native dev hire.

Recommended Starting Point: Scenario C — Hybrid (Lovable/Cursor + n8n + Supabase + Vertex AI ADK). Maximizes development speed with tools you know, preserves a clear migration path to GCP-native as scale demands it, and avoids the lock-in risk of a fully low-code stack. For Phase 2 iOS, pivot to FlutterFlow consuming the same Supabase/GCP backend.

3. Layer-by-Layer Stack Analysis

For each layer of the platform, the options are compared with cost estimates, tradeoffs, and a recommendation anchored to TradesOn's context.

3.1 Frontend — Web MVP

Tool	What It Is	Estimated Cost	Pros	Cons	TradesOn Fit
Lovable	AI-powered React app generator. Describe UI in plain language, get working code.	$0 (hobby) / $25/mo (Pro)	Fastest UI prototyping possible. Real React code exported. Great for non-devs.	Complex custom logic requires code edits. No native app output.	★★★ RECOMMENDED for Web MVP. Use for all customer-facing screens (S-01 to S-32). Export code to Cursor for custom logic layers.
Cursor	AI-assisted code editor (VS Code fork). Write and edit real React/Next.js code with AI help.	$20/mo per dev	Full code control. Best for custom business logic, API integrations, and edge cases Lovable can't handle.	Slower than Lovable for UI scaffolding. Requires coding skill.	★★★ RECOMMENDED as Lovable's companion. Use Cursor for AI agent integration, payment flows, and admin portal screens.
MindStudio	No-code AI app and workflow builder. Strong for building AI chat flows and agent UIs.	$0 (free) / $49+/mo	Zero code for AI interaction UIs. Good for rapid agent prototyping.	Not a full-stack web app platform. Limited to AI-centric UIs. Hard to customize deeply.	★★ Use for internal AI workflow prototyping and MindStudio-hosted Compliance Agent UI only. Not for primary web app screens.
Antigravity	Low-code web app builder (similar to Webflow/Bubble but more developer-oriented).	Pricing varies	Visual builder, some logic capabilities.	Less community support than alternatives. Unclear long-term roadmap.	★ Not recommended as primary tool given Lovable covers the same ground with better AI integration.
Next.js (custom)	React framework for production web apps. Full code control.	$0 (open source) + hosting	Maximum flexibility. Production-grade. Native Vercel deployment.	Requires experienced React developer. Slower to scaffold.	★★ Reserve for Phase 1.5 if Lovable-exported code needs significant refactoring before scale.

3.2 Frontend — Native iOS (Phase 2 Beta)

Tool	What It Is	Estimated Cost	Pros	Cons	TradesOn Fit
FlutterFlow	Visual Flutter app builder. Design and logic in browser, exports native Flutter/Dart code.	$0 (free) / $70/mo (Pro) / $400/mo (Teams)	Fastest path to iOS without a native dev. Real Flutter code export. Good Supabase + Firebase integrations. Active ecosystem.	Pro-level customization requires Dart knowledge. Export code quality varies. Per-seat cost scales.	★★★ RECOMMENDED for Phase 2 iOS Beta. Connects directly to your Supabase backend. Handles maps, camera, push notifications natively.
React Native (Expo)	JavaScript-based cross-platform mobile framework.	$0 (open source)	Reuses web React knowledge. Large community. OTA updates via Expo.	Performance ceiling lower than Flutter. Some native modules need native bridge work.	★★ Strong alternative if your team has React expertise from Lovable/Cursor Phase 1. Slower to first build than FlutterFlow.
Swift / UIKit (native)	Apple's native iOS language and framework.	$99/yr Apple dev account	Best performance and platform integration. No abstraction layer.	Requires dedicated iOS developer. Highest cost to hire.	★ Not recommended for MVP team. Revisit post-Series A if performance demands it.

3.3 Backend, Database & Orchestration

Tool	Layer	Estimated Cost	Pros	Cons / Risks	TradesOn Fit
Supabase	Database + Auth + Storage + Realtime	$0 (free) / $25/mo (Pro) / $599+/mo (Team)	Postgres under the hood — production-grade. Built-in auth, storage, realtime subscriptions. Already in HomeDeetz stack. Lovable and FlutterFlow have native connectors.	Free tier has connection limits and pauses. Heavy realtime (live tracking) may need supplements.	★★★ RECOMMENDED as primary database for both phases. Use Supabase for all structured data: users, jobs, quotes, payments, compliance records.

GCP Cloud SQL (Postgres)	Managed Postgres on GCP	$30-120/mo (db-f1-micro to db-g1-small)	Fully managed Postgres. Deep GCP integration for Vertex AI workflows. No connection limits.	Higher baseline cost than Supabase free tier. Requires more configuration. VPC instability risk you flagged is real on VM-backed instances.	★★ Use as a migration target when Supabase hits limits at scale (10K+ users). Not recommended for MVP start.
Firebase Firestore	NoSQL realtime database (GCP)	$0 (Spark) / Pay-as-you-go	Best-in-class realtime sync. Native Flutter/FlutterFlow integration. No server needed for reads.	NoSQL schema is harder to query for complex reporting (compliance audit logs, invoice line items). Migration from Postgres is painful.	★★ Use Firebase Firestore specifically for live job tracking (real-time GPS updates) as a supplemental layer only. Not as primary database.
n8n (self-hosted or cloud)	Workflow orchestration / automation	$0 (self-hosted) / $20+/mo (cloud)	Visual workflow builder. Excellent for multi-step compliance pipelines, email/SMS triggers, webhook handling. Already in HomeDeetz.	Self-hosted on GCP Cloud Run can have cold-start delays. Cloud plan costs rise with execution volume.	★★★ RECOMMENDED for all automation workflows: compliance verification triggers, job status notifications, invoice delivery, activation emails. Replaces need for custom backend workers.
GCP Cloud Run	Serverless container hosting	Pay-per-request (~$0-30/mo at MVP scale)	No VPC instance management — solves your VM instability issue. Auto-scales to zero. Ideal for n8n and API backends.	Cold starts (mitigated with min-instances). Requires Docker knowledge.	★★★ RECOMMENDED for hosting n8n and any custom API microservices. Eliminates the VM/VPC instability problem you experienced.
GCP Cloud Functions	Serverless event-driven functions	Pay-per-invocation (very cheap at MVP)	Zero infrastructure. Ideal for webhooks (Stripe, compliance API callbacks).	Stateless — not for long-running processes. 9-min timeout limit.	★★ Use for Stripe webhook handlers, compliance API callbacks, and event-driven triggers. Not for the main API.

3.4 AI Agent Layer — Job Intake, Compliance, Routing

Tool	What It Does	Estimated Cost	Pros	Cons	TradesOn Fit

Vertex AI ADK(Agent Dev Kit)	GCP's framework for building multi-step AI agents with tool use, memory, and orchestration.	Compute cost only (~$0.002-0.02/1K tokens via Gemini). ADK itself is free.	Purpose-built for production AI agents. Integrates natively with GCP services. Supports multi-agent pipelines (Intake Agent + Compliance Agent).	Requires GCP knowledge to deploy. Steeper setup than MindStudio.	★★★ RECOMMENDED for Compliance Agent and Job Intake Agent in both phases. Your GCP familiarity makes this viable. Use Gemini 2.0 Flash for cost efficiency.
Vertex AI Gemini API(direct)	Call Gemini models directly via REST API or SDK.	$0.075-0.15/1M input tokens (Flash). $1.25/1M (Pro).	Cheapest path to Gemini. Simple to integrate. Good vision capability for job photos.	No agent orchestration built in — you handle state yourself.	★★★ Use for the Job Intake Agent's NLP/vision processing. Flash model keeps costs very low at MVP scale.
OpenAI GPT-4o API	OpenAI's multimodal flagship model.	$2.50/1M input tokens (GPT-4o). $0.15 for GPT-4o-mini.	Best-in-class reasoning. Excellent for job summary drafting. Large ecosystem.	Not on GCP — adds a second vendor. Higher cost than Gemini Flash.	★★ Strong alternative if Gemini quality is insufficient for job summary drafting. Benchmark both on 20 sample job inputs before committing.
MindStudio	No-code AI workflow builder for building GPT-based chatbots and simple agents.	$0 (free) / $49+/mo (Pro)	Zero code. Fast to prototype. Good for simple Q&A agents.	Not designed for multi-step compliance pipelines. Limited tool-calling. Hard to integrate with Supabase/GCP programmatically.	★ Use only for internal demos or simple FAQ-style chatbots. Do not rely on it for the production Compliance Agent or Job Intake Agent.
n8n AI Nodes(LangChain integration)	n8n has built-in LLM nodes that can chain AI calls within automation workflows.	Included in n8n cost	Reuses your existing n8n setup. Can trigger AI calls as part of compliance or notification workflows.	Less capable than ADK for complex multi-step agent reasoning.	★★ Use as a bridge: n8n triggers the Vertex AI ADK agent, collects results, and routes to Supabase or sends notifications.

3.5 Monthly Cost Estimate by Scenario — MVP Scale (<500 Users)

Cost Component	Scenario AFull Low-Code	Scenario BGCP Native	Scenario CHybrid (Rec.)	Scenario DFull GCP	Scenario EFlutterFlow First	Scenario FCursor + Supabase	Notes

Lovable (Pro)	$25	$0	$25	$0	$0	$0	1 seat
Cursor	$0	$0	$20	$0	$0	$20	1 seat
MindStudio	$49	$0	$0	$0	$49	$0	If used
FlutterFlow	$0	$0	$0	$0	$70	$0	Phase 2
Supabase	$25	$0	$25	$0	$25	$25	Pro plan
GCP Cloud Run / Functions	$0	$30	$15	$30	$0	$0	At MVP scale
GCP Cloud SQL	$0	$60	$0	$60	$0	$0	db-f1-micro
Firebase (Realtime only)	$0	$10	$5	$10	$10	$0	Tracking layer
n8n (Cloud Starter)	$20	$0	$20	$0	$0	$20	Or self-host free
Vertex AI (Gemini Flash)	$5	$5	$5	$5	$5	$5	~1K job analyses/mo
Stripe	$0	$0	$0	$0	$0	$0	% per transaction
Twilio + SendGrid	$15	$15	$15	$15	$15	$15	SMS + email
Google Maps API	$10	$10	$10	$10	$10	$10	Geocoding + embed
TOTAL / MONTH	**~$149**	**~$130**	**~$140**	**~$130**	**~$184**	**~$95**	Estimates only

4. Pre-TDD Discovery: Decisions, Questions & Dependencies

The following questions and decisions MUST be resolved before the TDD is drafted. Each unresolved item represents a risk of rework during engineering. They are organized by platform layer and delivery phase.

4.1 Architecture & Hosting Decisions

Phase 1 — Web MVP

DECISION NEEDED: *Will the backend API be a dedicated Node.js/Express service or Next.js API routes (co-located with frontend)? This affects deployment topology and how Lovable-exported code integrates with backend logic.*

DISCOVERY Q: *Does the team have capacity to manage a Cloud Run deployment, or should the backend be hosted on Vercel/Railway for simplicity in Phase 1?*

DEPENDENCY: *Hosting decision must be made before any backend service is scaffolded. Lovable exports React — you need to decide where that React app is served from (Vercel is the default and simplest).*

RISK: *If Lovable-exported code is heavily customized in Cursor, re-importing into Lovable for future UI changes breaks the workflow. Establish a one-way export policy early.*

Phase 2 — Native iOS

DECISION NEEDED: *Will FlutterFlow connect directly to Supabase, or will it exclusively consume the same REST API built for the web app? Direct Supabase connection is faster to build but creates two data access patterns to maintain.*

DISCOVERY Q: *Does the iOS app need offline capability for job checklists (S-35) and route view (S-36)? If yes, local SQLite (via FlutterFlow) needs to be in scope from the start.*

DEPENDENCY: *iOS App Store submission requires an Apple Developer Account ($99/yr) and a macOS machine for final build signing. Confirm this is available before starting Phase 2.*

RISK: *FlutterFlow's code export quality degrades with complex custom logic. Define the boundary early: what stays in FlutterFlow UI vs. what is handled by custom Dart code or API calls.*

4.2 Database & Data Architecture Decisions

Phase 1 — Web MVP

DECISION NEEDED: *Will you use Supabase Row-Level Security (RLS) to enforce role-based data access (homeowner can't see another homeowner's jobs), or handle this at the API layer? RLS is more secure but adds schema complexity.*

DISCOVERY Q: *How many concurrent jobs do you expect at MVP launch? Supabase free tier pauses after 1 week of inactivity and has a 500MB database limit. Is the $25/mo Pro plan budgeted from day one?*

DEPENDENCY: *The Supabase schema must be designed before any screen development begins. Schema decisions (especially job statuses, user roles, compliance states) cascade into every function in the PRD.*

DECISION NEEDED: *Will job status history be append-only (event log) or mutable (single status field updated)? Event sourcing gives you a full audit trail (required for S-20 Audit Log) but requires more careful schema design.*

RISK: *If you start with a mutable status field and later need an audit log, retrofitting is expensive. Design for append-only status events from the start.*

Phase 2 — Native iOS

DECISION NEEDED: *Will Firebase Realtime Database be used as a supplemental layer for live GPS tracking (S-37), or will Supabase Realtime handle it? Firebase is more battle-tested for high-frequency location updates; Supabase Realtime is sufficient for moderate update rates (5-10 sec).*

DISCOVERY Q: *What is the expected number of concurrent active jobs during peak hours? This determines whether Supabase Realtime is sufficient or Firebase is needed.*

4.3 AI Agent Design Decisions

Phase 1 — Web MVP

DECISION NEEDED: *Will the Job Intake Agent (Function 6) run synchronously (user waits for AI response) or asynchronously (user submits, gets notified when AI analysis is complete)? Synchronous is simpler to build; async is better UX for slow networks.*

DISCOVERY Q: *What is the acceptable latency for AI job analysis? If target is <5 seconds, Gemini Flash is appropriate. If complex photo analysis is needed, Gemini Pro may be required at higher cost.*

DECISION NEEDED: *Will the Compliance Agent (Function 4) have access to real third-party verification APIs (license lookup, insurance check) at MVP, or will it use a manual review fallback with AI-assisted document reading?*

DEPENDENCY: *Third-party compliance API contracts (Verisk, ACORD, or state license APIs) must be signed before the Compliance Agent can be built. This is on the critical path for tradesperson onboarding.*

DISCOVERY Q: *What happens when the AI Intake Agent has low confidence in a job classification? Define the fallback: re-prompt the user, route to manual review, or allow submission with a flag.*

RISK: *If MindStudio is used for any agent that touches Supabase data, you will need to build a custom API layer between them. MindStudio does not have a native Supabase connector as of 2025.*

Phase 2 — Native iOS

DECISION NEEDED: *Will AI-powered features (job intake, voice input) run server-side or leverage on-device models (Core ML / Gemini Nano)? On-device is faster and works offline but limits model capability.*

DISCOVERY Q: *Is voice-to-text job intake (S-26) in scope for Phase 2 iOS? If yes, decide between browser MediaRecorder (web), Google Speech-to-Text API (cloud), or iOS native SFSpeechRecognizer.*

4.4 Payment Architecture Decisions

Phase 1 — Web MVP

DECISION NEEDED: *Will Stripe Connect use Standard or Express accounts for tradesperson payouts? Standard gives tradespeople a full Stripe dashboard; Express is simpler to onboard but less flexible. Express is recommended for MVP.*

DECISION NEEDED: *What is the platform fee percentage and when is it deducted — at payment capture or at payout? This affects how you configure Stripe's application fee vs. manual split logic.*

DEPENDENCY: *Stripe account must be created, Connect enabled, and test mode configured before S-45 or S-46 development begins. Stripe's Connect onboarding review can take 1-2 days.*

DISCOVERY Q: *Will the platform hold funds in escrow between job completion and customer approval, or release immediately on completion mark? Escrow requires a more complex PaymentIntent flow.*

RISK: *Pre-authorization holds (captured at quote acceptance) expire after 7 days on most card networks. If a job runs long, the hold must be re-captured. Define the maximum job duration and build an auto-recapture workflow.*

Phase 2 — Native iOS

DECISION NEEDED: *Will Apple Pay be supported in the iOS app? If yes, Stripe's native iOS SDK supports this but requires Apple Pay merchant certificate setup via your Apple Developer account.*

DEPENDENCY: *Apple's App Store Review guidelines require in-app purchases of digital goods to use Apple's IAP system. Confirm that TradesOn's payment model (service marketplace) qualifies for external payment processing (it should, but verify with legal).*

4.5 Real-Time Tracking & Notifications Decisions

Phase 1 — Web MVP

DECISION NEEDED: *For live tracking (S-37), will the web app use polling (simpler, 5-sec refresh via setInterval) or Supabase Realtime subscriptions (WebSocket-based, lower latency)? Polling is recommended for MVP given lower complexity.*

DECISION NEEDED: *For push notifications on web, will you use browser Push API (requires HTTPS and user permission grant) or fall back to email/SMS-only for MVP? Browser push has low opt-in rates on desktop. Email + SMS is more reliable for MVP.*

DISCOVERY Q: *What is the required location update frequency for live tracking? Every 5 seconds covers most use cases. Every 1 second requires a different architecture (Firebase RTDB or direct WebSocket).*

Phase 2 — Native iOS

DECISION NEEDED: *Will push notifications use APNs (Apple Push Notification Service) directly via GCP Firebase Cloud Messaging, or through a managed service like OneSignal? Firebase Cloud Messaging is recommended given your GCP alignment.*

DEPENDENCY: *APNs certificates must be generated from your Apple Developer account before any push notification development begins. This is a Phase 2 blocker.*

4.6 Compliance & Verification Architecture Decisions

Phase 1 — Web MVP

DECISION NEEDED: *Will identity verification (across all user roles) use Stripe Identity, Persona, or Jumio? Stripe Identity is cheapest if already using Stripe ($1.50/verification). Persona is more flexible but higher cost.*

DECISION NEEDED: *Will document storage for compliance (IDs, licenses, insurance certs) be in Supabase Storage or GCP Cloud Storage? Supabase Storage is simpler; GCP offers better integration with Vertex AI for document analysis.*

DISCOVERY Q: *What is the SLA for compliance review? If a tradesperson submits documents on Monday, when are they expected to be verified? Define the target (e.g., 24 hours) — this determines whether full AI automation is required at MVP or whether a human review queue is acceptable.*

DEPENDENCY: *The compliance verification workflow (n8n pipeline + Vertex AI document reading + third-party APIs) must be fully specced before S-07, S-08, and S-17 can be built. These screens are blocked until the backend workflow is defined.*

RISK: *If a third-party license verification API is not available for Illinois (or your target launch state), you need a manual review fallback ready at launch. Do not block tradesperson onboarding on full automation.*

4.7 Phase Sequencing & Critical Path

The following items are on the critical path for each phase. Nothing downstream can begin until these are resolved.

#	Decision / Dependency	Blocks These Screens	Phase	Recommended Owner
1	**Supabase schema designed and seeded (users, jobs, quotes, compliance, payments)**	All screens in Functions 1-12	**Phase 1 — Web MVP**	Engineering Lead — must be complete before any backend dev

2	**Stripe account + Connect (Express) enabled in test mode**	S-45, S-46, S-47 (Payments)	**Phase 1 — Web MVP**	Product Lead / Engineering
3	**Compliance API vendor selected (or manual fallback confirmed)**	S-07, S-08 (Trade Onboarding), S-17 (Compliance Review)	**Phase 1 — Web MVP**	Product Lead + Legal
4	**Identity verification vendor selected (Stripe Identity recommended)**	S-03 through S-08 (all onboarding)	**Phase 1 — Web MVP**	Engineering Lead
5	**AI model benchmarked: Gemini Flash vs. GPT-4o-mini on 20 sample jobs**	S-26, S-27, S-28 (AI Intake)	**Phase 1 — Web MVP**	Engineering + Product
6	**n8n hosted on Cloud Run or Cloud plan — environment ready**	Compliance Agent, notification workflows	**Phase 1 — Web MVP**	Engineering Lead
7	**Job expiration window defined (recommended: 72 hours)**	S-29 through S-32 (Job Board)	**Phase 1 — Web MVP**	Product Lead
8	**Platform fee % defined and Stripe application fee configured**	S-42, S-44, S-46 (Invoicing + Payouts)	**Phase 1 — Web MVP**	Business / Product Lead
9	**Apple Developer Account active + FlutterFlow Pro plan**	All Phase 2 iOS screens	**Phase 2 — iOS Beta**	Product Lead
10	**APNs certificates generated and configured in Firebase Cloud Messaging**	Push notifications across Phase 2	**Phase 2 — iOS Beta**	Engineering Lead
11	**FlutterFlow ↔ Supabase connection strategy confirmed (direct vs. API)**	All Phase 2 screens	**Phase 2 — iOS Beta**	Engineering Lead — must decide before FlutterFlow build begins
12	**Apple Pay merchant certificate configured (if in scope)**	S-45 (iOS Payment)	**Phase 2 — iOS Beta**	Engineering Lead

4.8 Addressing the GCP VPC Instability You Experienced

You flagged that working with GCP VPC/VM instances caused 'random changes' — this is a known pain point with Compute Engine VMs and complex VPC networking rules. The good news: the recommended Scenario C stack avoids this almost entirely.

Component	Instead of VM/VPC...	Use This
n8n hosting	Compute Engine VM with VPC firewall rules	GCP Cloud Run (serverless container) — no VMs, no VPC config, no random state changes. Auto-scales. Costs ~$0 at idle.
Backend API	App Engine Standard or GCE VM	Cloud Run or Vercel (for Next.js). Zero infrastructure management.
Database	Cloud SQL on VM-backed instance	Supabase (managed Postgres) for MVP. No VPC required. Cloud SQL is fine but use it without private VPC networking for simplicity.
AI Agents	VM-hosted Python process	Vertex AI ADK on Cloud Run or Cloud Functions — fully serverless, no persistent instance.
File storage	GCS bucket with complex IAM + VPC rules	Supabase Storage (simple bucket-style with built-in RLS) for MVP. GCS for Phase 2 if needed.

Bottom line: The VPC instability you experienced was a symptom of using persistent VM-based infrastructure. Moving to Cloud Run (serverless containers) and Supabase eliminates that class of problem entirely. You get GCP's AI and analytics capabilities without managing the networking layer.

5. Recommended Architecture Summary (Scenario C)

The following summarizes the recommended end-state architecture for TradesOn across both phases. This serves as the narrative input to the architecture diagram the engineering team will produce in the TDD.

Layer	Phase 1 — Web MVP	Phase 2 — iOS Beta (additions/changes)
UI / Frontend	Lovable (React) for customer-facing screens. Cursor for admin portal and custom logic screens. Hosted on Vercel.	FlutterFlow (Flutter/Dart) for iOS app. Consumes same REST API as web. Deployed to App Store.
API Layer	Next.js API Routes or Express on Cloud Run. JWT auth via Supabase Auth. REST endpoints for all PRD functions.	Same API. Add WebSocket endpoint for live tracking if Supabase Realtime is insufficient.
Database	Supabase (Postgres). RLS for role-based access. Supabase Storage for compliance documents.	Same. Add Firebase Realtime DB supplemental layer for high-frequency GPS updates only.
Automation / Orchestration	n8n on Cloud Run. Workflows for: compliance triggers, job notifications, invoice delivery, activation emails.	Same. Add FlutterFlow webhook triggers where needed.
AI Agents	Vertex AI ADK (Gemini Flash) for Job Intake Agent and Compliance Document Agent. Invoked via n8n or direct API call.	Same agents. Add on-device voice-to-text (iOS SFSpeechRecognizer) for job input.
Payments	Stripe PaymentIntents + Stripe Connect Express. Stripe.js on web. Webhooks handled by Cloud Functions.	Stripe iOS native SDK. Apple Pay if in scope.
Maps / Location	Google Maps JavaScript API (embedded in web). 5-second polling for live tracking.	Google Maps SDK for Flutter. Firebase RTDB or Supabase Realtime for live tracking WebSocket.
Notifications	SendGrid (email) + Twilio (SMS) via n8n workflows. Browser push notifications (best effort).	Firebase Cloud Messaging (APNs) for native iOS push. Same email/SMS via n8n.
Auth	Supabase Auth (email + password). JWT sessions.	Same. Add iOS biometric layer (Face ID) via FlutterFlow local auth plugin.
Admin / Internal	Cursor-built React admin portal (S-17 through S-23). Hosted on Vercel (access-restricted).	No changes — admin is web-only.

Next Steps to TDD

1. Circulate this document to engineering and product leads for decision sign-off on all items in Section 4.
2. Resolve the 12 critical path items in Section 4.7 — assign an owner and due date to each.

3. Benchmark AI models: run 20 sample job descriptions through Gemini Flash and GPT-4o-mini and score accuracy of job classification, severity scoring, and summary quality.
4. Set up the Supabase project and design the core schema (users, jobs, quotes, compliance_documents, payment_intents, audit_log) — this unblocks all other engineering work.
5. Once all decisions in Section 4 are resolved, the TDD author can map each decision to a concrete technical spec with confidence.

Estimated time to TDD readiness: 2–3 weeks if Section 4 decisions are driven to resolution in parallel. The database schema design and Stripe account setup are the longest poles.